

Machine Learning Theory Notes

Extracted from handwritten notes

Linear Regression (OLS)

Contents

1	Linear Regression Using OLS (Ordinary Least Squares)	3
1.1	Model Equation	3
1.2	Cost Function (Loss)	3
1.3	Analytical Solution (OLS Formula)	3
2	Multiple Linear Regression	4
2.1	Model Equation	4
2.2	Cost Function	4
3	Regression Performance Metrics	5
3.1	Mean Squared Error (MSE)	5
3.2	Mean Absolute Error (MAE)	5
3.3	Root Mean Squared Error (RMSE)	5
3.4	R-Squared (R^2)	5
3.5	Adjusted R-Squared	5
4	Overfitting and Underfitting	7
4.1	Bias and Variance Tradeoff	7
5	Regularization Techniques	8
5.1	Ridge Regression (L2 Regularization)	8
5.2	Lasso Regression (L1 Regularization)	8
5.3	ElasticNet Regression	8
6	Cross Validation (CV)	9
6.1	Types of Cross Validation	9
7	Classification Fundamentals	10
7.1	Logistic Regression (Binary Classification)	10
7.1.1	The Sigmoid (activation) Function	10
7.1.2	Cost Function (Log Loss / Binary Cross-Entropy)	10

8	Classification Performance Metrics	11
8.1	Confusion Matrix	11
8.2	Accuracy	11
8.3	Precision	11
8.4	Recall (Sensitivity)	11
8.5	F-Beta Score	12
8.6	ROC and AUC Curve	12
9	Polynomial Regression	13
9.1	Model Equation	13
10	Support Vector Machines (SVM)	15
10.1	Model Equations	15
10.2	The Kernel Trick	15
11	Naive Bayes	16
11.1	Bayes Theorem	16
11.2	Naive Bayes Assumption Formula	17
12	K-Nearest Neighbors (KNN)	18
12.1	Distance Metrics	18
13	Decision Trees	19
13.1	Splitting Criteria & Purity Metrics	19
13.2	Information Gain	20
13.3	Handling Numerical / Continuous Features	20
13.4	Decision Tree Regression (CART)	20
14	Random Forest	21
14.1	Key Concepts	21
15	Boosting Algorithms	22
15.1	AdaBoost (Adaptive Boosting)	22
15.2	Gradient Boosting	23
15.3	XGBoost (Extreme Gradient Boosting)	23
16	Unsupervised Learning: Dimensionality Reduction	24
16.1	Principal Component Analysis (PCA)	24
17	Unsupervised Learning: Clustering	25
17.1	K-Means Clustering	25
17.2	Hierarchical Clustering	25
17.3	DBSCAN (Density-Based Spatial Clustering of Applications with Noise)	26
17.4	Silhouette Score (Clustering Validation)	26
18	Anomaly Detection	27
18.1	Isolation Forest	27
18.2	Local Outlier Factor (LOF)	27

1 Linear Regression Using OLS (Ordinary Least Squares)

Intuition: Linear Regression models the relationship between a dependent variable (y) and one or more independent variables (x) by fitting a straight line to the observed data. The goal is to find the best possible line that minimizes the total error between predicted and actual values.

1.1 Model Equation

The hypothesis function represents our prediction line:

$$h(x) = \beta_0 + \beta_1 x \implies \hat{y} \quad (1)$$

Where:

- β_0 : Intercept (Bias)
- β_1 : Slope (Coefficient/Weight)
- $\hat{y}$: Predicted value

1.2 Cost Function (Loss)

To define what makes a line "best fit", we measure the error using Mean Squared Error (MSE). We want to find the parameters (β_0, β_1) that result in the **global minimum** of this cost function.

$$S(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \quad (2)$$

1.3 Analytical Solution (OLS Formula)

Using calculus (setting partial derivatives to zero), the final exact formulas to find the optimal intercept and slope are:

1. Coefficient (Slope):

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (3)$$

2. Intercept (Bias):

$$\beta_0 = \bar{y} - \beta_1 \bar{x} \quad (4)$$

Note: $\bar{x}$ and $\bar{y}$ represent the mean of x and y respectively.

2 Multiple Linear Regression

Intuition: An extension of Simple Linear Regression where we use **multiple** independent features $(x_1, x_2, \dots, x_n)$ to predict a single continuous output $(\hat{y})$.

For example, predicting House Price based on number of rooms, size, and location.

2.1 Model Equation

The prediction line becomes a hyperplane in higher dimensions:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n \implies \hat{y} \quad (5)$$

Where:

- θ_0 : Intercept (Bias)
- $\theta_1, \theta_2, \dots, \theta_n$: Coefficients (Weights for each feature)

2.2 Cost Function

The objective remains the same: find the global minimum of the error surface $J(\theta_0, \theta_1, \dots, \theta_n)$ using Gradient Descent or OLS.

3 Regression Performance Metrics

3.1 Mean Squared Error (MSE)

Measures the average of the squares of the errors. It is a convex, differentiable function that penalizes large outliers heavily.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (6)$$

- **Pro:** Differentiable, converges faster to global minima.
- **Con:** Not robust to outliers. Units are squared, making interpretation unintuitive.

3.2 Mean Absolute Error (MAE)

Measures the average of the absolute differences between predictions and actual values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (7)$$

- **Pro:** Robust to outliers. Values are in the same unit as the target variable.
- **Con:** Not strictly differentiable at zero (subgradients are needed), making optimization complex.

3.3 Root Mean Squared Error (RMSE)

The square root of MSE. It brings the unit of error back to the original unit of the target variable.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (8)$$

- **Pro:** Same unit as target. Differentiable.
- **Con:** Still heavily influences by outliers (since underlying formula squares the error).

3.4 R-Squared (R^2)

- **Intuition:** Known as the Coefficient of Determination. Indicates the goodness of fit of a set of predictions to the actual values. Value generally between 0 and 1. If $R^2 \approx 1$, the model is very accurate.
- **Equation:** $R^2 = 1 - \frac{SS_{res}}{SS_{total}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$
- **Problem with R^2 :** As the number of independent features increases, R^2 always increases or remains the same, even if the new features are completely unrelated to the target variable.

3.5 Adjusted R-Squared

- **Intuition:** Penalizes the model for adding features that do not improve the model. Unlike R^2 , Adjusted R^2 will decrease if an irrelevant feature is added.

- **Equation:** $R_{adj}^2 = 1 - \frac{(1-R^2)(n-1)}{n-p-1}$
- **Where:** n = number of data points, and p = number of independent features.

4 Overfitting and Underfitting

To build generalized machine learning models, we split our data into **Training** (e.g., 70%) and **Testing** (e.g., 30%) datasets.

4.1 Bias and Variance Tradeoff

- **Generalized Model:** Performs well on BOTH Train and Test data.
 - High Train Accuracy (Low Bias)
 - High Test Accuracy (Low Variance)
- **Overfitting:** The model memorizes the training data but fails on unseen data.
 - High Train Accuracy (Low Bias)
 - Low Test Accuracy (**High Variance**)
- **Underfitting:** The model is too simple to capture patterns in the data at all.
 - Low Train Accuracy (**High Bias**)
 - Low Test Accuracy (**High Variance**)

5 Regularization Techniques

Regularization is used to add a penalty to the Cost Function to **reduce Overfitting** (high variance) by making the model simpler (shrinking the coefficients).

5.1 Ridge Regression (L2 Regularization)

Goal: Reduce Overfitting by shrinking the coefficients of less important features close to zero.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n \theta_j^2 \quad (9)$$

Where:

- $\lambda \sum_{j=1}^n \theta_j^2$ is the **L2 Penalty** (Sum of squared weights).
- λ is the hyperparameter. If $\lambda \rightarrow \infty$, slope becomes ≈ 0 .

5.2 Lasso Regression (L1 Regularization)

Goal: Feature Selection. It reduces Overfitting by shrinking the coefficients of less important features exactly to **zero**.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n |\theta_j| \quad (10)$$

Where:

- $\lambda \sum_{j=1}^n |\theta_j|$ is the **L1 Penalty** (Sum of absolute weights).
- Forces some θ_j directly to 0, acting as automatic feature selection.

5.3 ElasticNet Regression

Goal: Combines both Ridge and Lasso for both reducing overfitting and performing feature selection.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda_1 \sum_{j=1}^n \theta_j^2 + \lambda_2 \sum_{j=1}^n |\theta_j| \quad (11)$$

- λ_1 : Penalty hyperparameter for Ridge (reduces overfitting).
- λ_2 : Penalty hyperparameter for Lasso (feature selection).

6 Cross Validation (CV)

Cross-validation is a resampling method used to evaluate machine learning models on a limited data sample. The goal is to test the model's ability to predict new, unseen data to flag problems like overfitting.

6.1 Types of Cross Validation

1. Leave One Out CV (LOOCV)

- Uses 1 data point for validation and the remaining $n - 1$ for training.
- **Pro:** Unbiased evaluation.
- **Con:** Very high training complexity. Requires training the model n times (n = total number of records).

2. Leave P Out CV

- Leaves P data points for validation and trains on the rest. Very computationally expensive.

3. K-Fold CV

- Splits data into K equal-sized folds.
- Trains the model K times, each time using 1 fold for validation and $K - 1$ folds for training.
- Final accuracy is the **average** of all K accuracies.
- Example: For 500 records and $K = 5$, the test size is $\frac{500}{5} = 100$ records each loop.

4. Stratified K-Fold CV

- An extension of K-Fold CV used mainly for Classification.
- Ensures that Each Fold has the exact same **ratio** of target classes as the original dataset (e.g., maintaining a 60:40 ratio of Class 1 to Class 0 in every single fold).

5. Time Series CV

- Used when data is temporally dependent (e.g., Stock Prices, Daily Sentiment Analysis).
- We cannot randomly shuffle or K-Fold time series data. We must respect the chronological order.
- **Method:** Train on $[Day1]$, Validate on $[Day2]$. Next split: Train on $[Day1, Day2]$, Validate on $[Day3]$, and so on.

7 Classification Fundamentals

7.1 Logistic Regression (Binary Classification)

Logistic Regression is used for binary classification (e.g., Output: Pass/Fail, 0/1).

Why can't we use Linear Regression for Classification?

1. **Outliers:** Linear regression tries to fit a straight line. If we have an extreme outlier, the best-fit line shifts significantly, changing the decision boundary and misclassifying points.
2. **Value Range:** Linear regression output can be > 1 or < 0 . For probabilities or binary classes, we want outputs strictly bounded between 0 and 1.

7.1.1 The Sigmoid (activation) Function

To fix the > 1 and < 0 problem, we wrap the linear regression equation in a Sigmoid function. This "squashes" the output to be between 0 and 1.

$$\text{Linear Equation: } z = \theta_0 + \theta_1 x_1 + \dots \quad (12)$$

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1 + e^{-z}} \quad (13)$$

$$\text{Logistic Hypothesis: } h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1)}} \quad (14)$$

Decision Boundary:

- If $h_{\theta}(x) \geq 0.5 \implies$ Class 1 (Pass)
- If $h_{\theta}(x) < 0.5 \implies$ Class 0 (Fail)

7.1.2 Cost Function (Log Loss / Binary Cross-Entropy)

We cannot use the Mean Squared Error (MSE) cost function for Logistic Regression because plotting MSE with the Sigmoid hypothesis results in a **Non-Convex** function (many local minima). To get a **Convex** function (one global minimum) for Gradient Descent, we use **Log Loss**.

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases} \quad (15)$$

Unified Cost Function Equation:

$$J(\theta_0, \theta_1) = -\frac{1}{2m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \quad (16)$$

Note: The goal remains to minimize $J(\theta)$ using Gradient Descent.

8 Classification Performance Metrics

8.1 Confusion Matrix

A table used to evaluate the performance of a classification model. It compares the **Actual Values** against the **Predicted Values**.

- **True Positive (TP)**: Predicted Positive (1), Actual Positive (1)
- **True Negative (TN)**: Predicted Negative (0), Actual Negative (0)
- **False Positive (FP)**: Predicted Positive (1), Actual Negative (0) → *Type 1 Error*
- **False Negative (FN)**: Predicted Negative (0), Actual Positive (1) → *Type 2 Error*

8.2 Accuracy

The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \quad (17)$$

Problem with Accuracy: It is not a reliable metric for **Imbalanced Datasets**. If a dataset has 90% Class A and 10% Class B, a dumb model predicting only Class A will blindly get 90% accuracy.

8.3 Precision

Out of all the predicted positive values, how many are actually positive? We use Precision when **False Positives (FP)** are a blunder / highly costly.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (18)$$

Use Case (Spam Classification): If a normal mail is classified as spam (False Positive), it's a huge blunder for the user because they miss important emails. Hence, we must optimize for high precision.

8.4 Recall (Sensitivity)

Out of all the actual positive values, how many did we correctly predict as positive? We use Recall when **False Negatives (FN)** are a blunder / highly costly.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (19)$$

Use Case (Disease/Diabetes Prediction): If a person actually has diabetes but the model predicts they don't (False Negative), it's a life-threatening blunder. The patient won't get treated. Hence, we must optimize for high recall in medical use cases.

Log Loss (Binary Cross-Entropy): For a single training example, the cost is:

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

The combined cost function for all m examples is:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

8.5 F-Beta Score

The F-Beta score is a generalization of the F1-score that allows us to give more weight to either Precision or Recall.

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

Common Beta Values:

- $\beta = 1$ (**F1-Score**): Harmonic mean of Precision and Recall. Used when both FP and FN are equally important.
- $\beta = 0.5$ (**F0.5-Score**): Gives more weight to **Precision**. Used when reducing False Positives (FP) is critical (e.g., Spam detection).
- $\beta = 2$ (**F2-Score**): Gives more weight to **Recall**. Used when reducing False Negatives (FN) is critical (e.g., Cancer detection).

8.6 ROC and AUC Curve

Intuition: The Receiver Operating Characteristic (ROC) curve is a graphical representation of a classification model's performance across all classification thresholds. It plots two parameters:

- **True Positive Rate (TPR):** Also known as Recall. Represents the proportion of actual positives correctly identified.
- **False Positive Rate (FPR):** Represents the proportion of actual negatives incorrectly identified as positives.

Final Formulas:

$$\text{TPR} = \frac{TP}{TP + FN} \quad (20)$$

$$\text{FPR} = \frac{FP}{FP + TN} \quad (21)$$

Area Under the ROC Curve (AUC): AUC measures the entire two-dimensional area underneath the ROC curve from (0, 0) to (1, 1). It provides an aggregate measure of performance across all possible classification thresholds. An AUC of 1.0 represents a perfect model, while 0.5 represents a random guess.

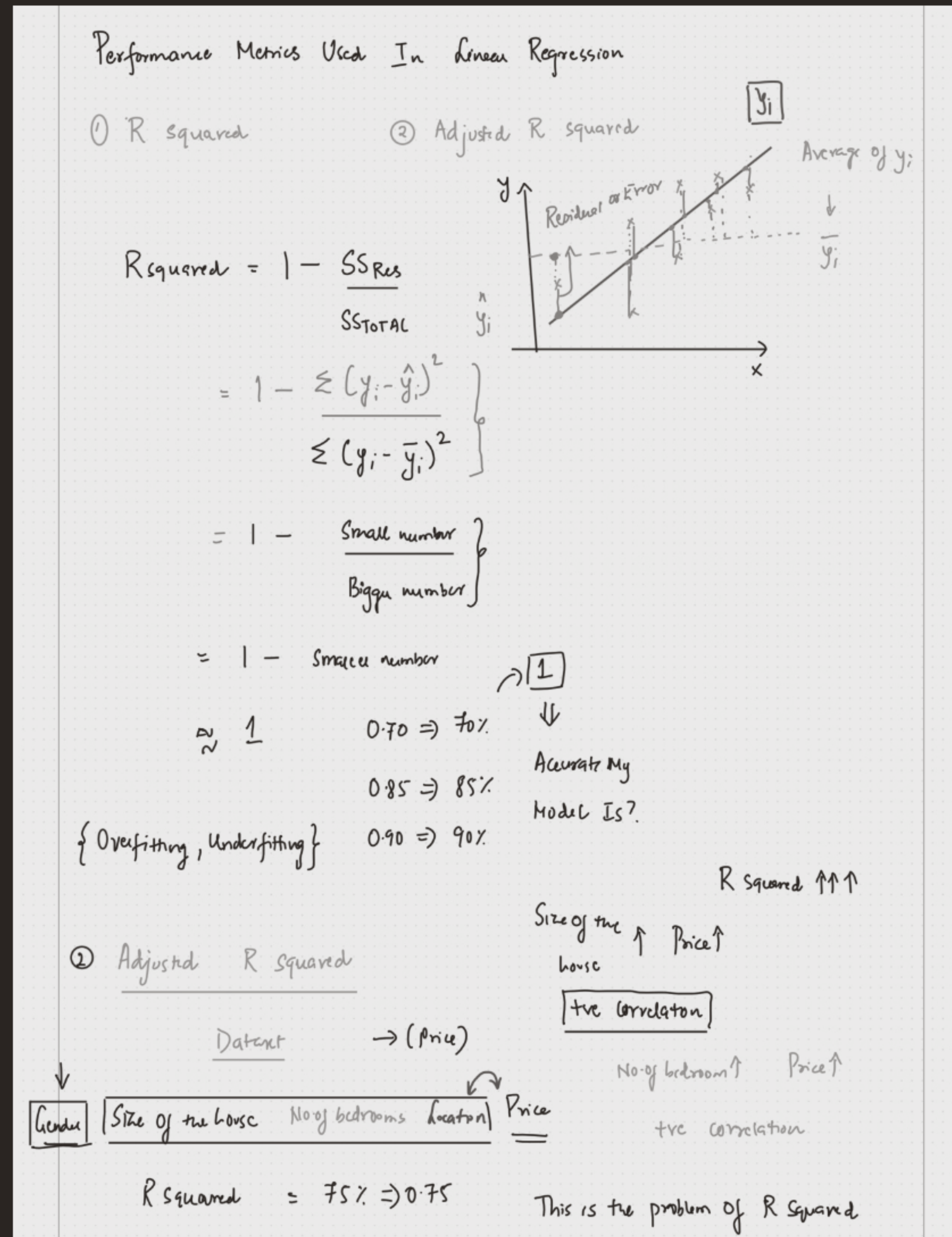


Figure 1: ROC Curve and AUC

9 Polynomial Regression

Intuition: Polynomial Regression is a form of linear regression in which the relationship between the independent variable x and the dependent variable y is modeled as an n th degree polynomial. We use it when the data points represent a **non-linear relationship** (e.g., a curve) that cannot be captured accurately by a simple straight line.

Even though it fits a non-linear model to the data, it is considered a special case of Multiple Linear Regression because the coefficients (θ) are still linear in relation to the output.

9.1 Model Equation

The hypothesis function for an n th degree polynomial is:

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots + \theta_n x^n \implies \hat{y} \quad (22)$$

Where:

- θ_0 : Intercept (Bias)

- $\theta_1, \theta_2, \dots, \theta_n$: Coefficients (Weights) for each polynomial term
- $x^2, x^3, \dots, x^n$: Higher degree polynomial features created from the original input x

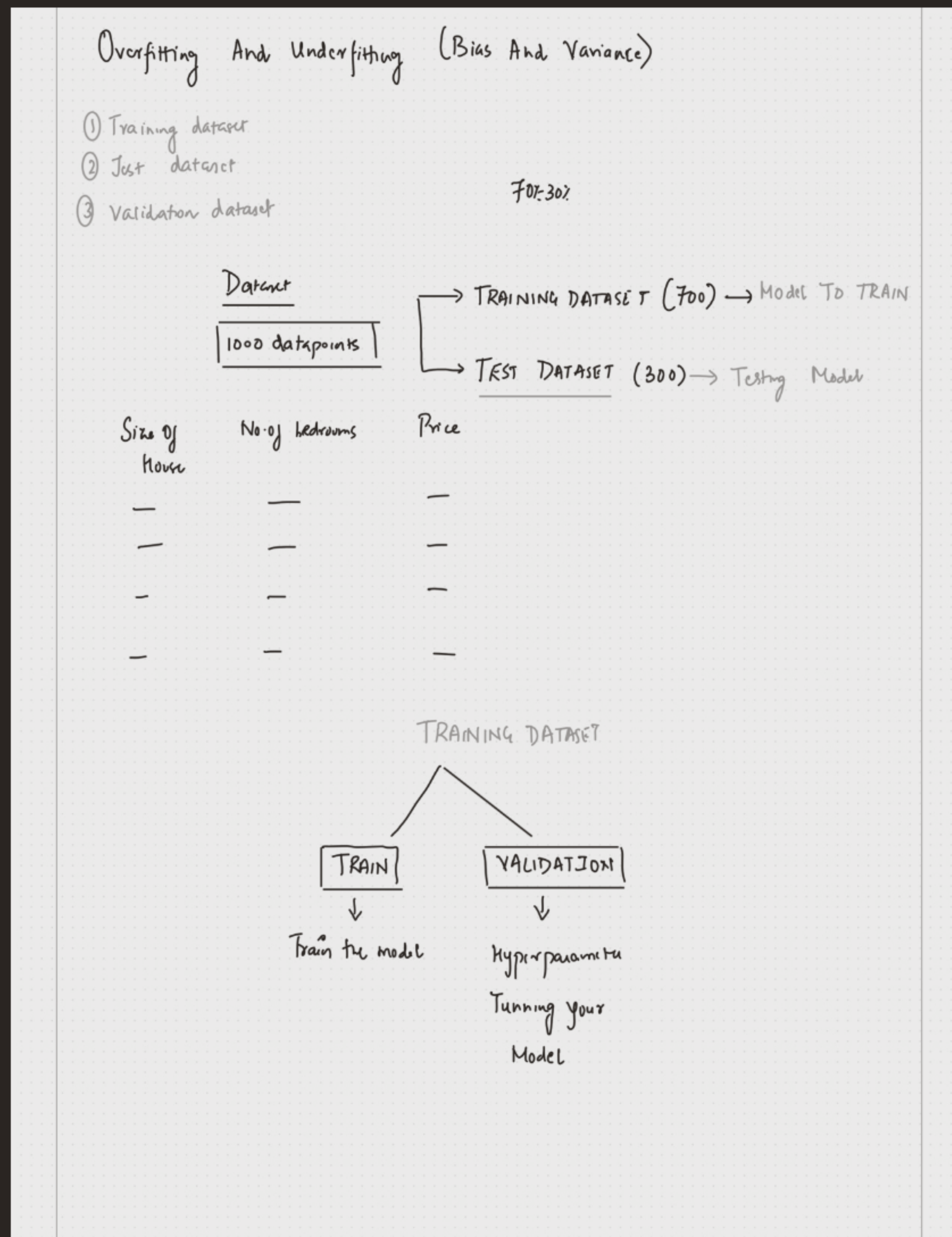


Figure 2: Polynomial Regression Curve Fitting

10 Support Vector Machines (SVM)

Intuition: SVM is a powerful classifier that finds the optimal hyperplane to separate classes while maximizing the **margin** (distance) between the closest data points of both classes (called Support Vectors).

- **Hard Margin:** Strict separation, no misclassifications allowed (highly sensitive to outliers).
- **Soft Margin:** Allows some misclassifications to achieve a better generalized model, controlled by the parameter C .

10.1 Model Equations

Hyperplane Equation:

$$w^T x + b = 0 \quad (23)$$

Soft Margin Optimization Objective:

$$\min_{w,b,\xi} \left(\frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \right) \quad (24)$$

Where ξ_i represents the slack variables allowing for margin violations, and C is the regularization parameter.

10.2 The Kernel Trick

Intuition: When data is not linearly separable in its original space, SVM uses the "Kernel Trick" to project the data into a higher-dimensional space where it becomes linearly separable, without explicitly computing the coordinates in that higher space.

- **Polynomial Kernel:** $K(x, x') = (\gamma \langle x, x' \rangle + r)^d$
- **RBF (Radial Basis Function) Kernel:** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$

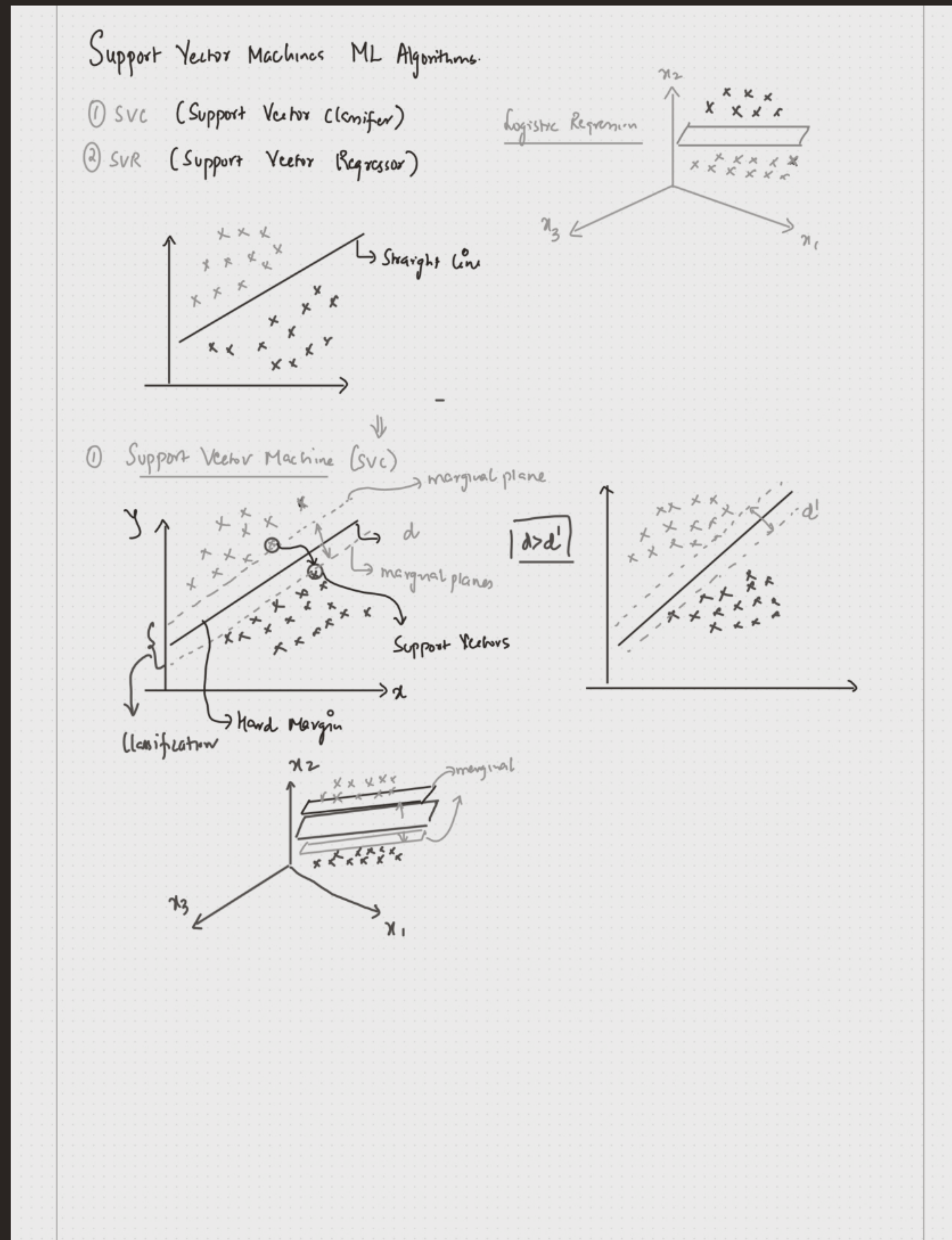


Figure 3: Support Vector Machine Margin and Hyperplane

11 Naive Bayes

Intuition: A probabilistic classification algorithm based on Bayes' Theorem. It is called "Naive" because it assumes that all features are perfectly independent of each other given the class label—a strong but highly effective assumption, especially for Text Classification (e.g., Spam Filtering).

11.1 Bayes Theorem

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)} \quad (25)$$

Where:

- $P(y|X)$: Posterior Probability (Probability of class y given features X)
- $P(X|y)$: Likelihood (Probability of features X given class y)
- $P(y)$: Prior Probability (Overall probability of class y)
- $P(X)$: Marginal Probability (Total probability of features X)

11.2 Naive Bayes Assumption Formula

By applying the conditional independence assumption for features $x_1, x_2, \dots, x_n$:

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y) \quad (26)$$

The predicted class $\hat{y}$ is the one that maximizes this product (Maximum A Posteriori).

Naive Bayes's Algorithm (Classification)

- ① Probability
- ② Bayes's Theorem

Independent Events

Rolling a Dice $\{1, 2, 3, 4, 5, 6\}$

$Pr(1) = \frac{1}{6}$ $Pr(2) = \frac{1}{6}$ $Pr(3) = \frac{1}{6}$

Dependent Events

① What is the probability of removing a orange marble and then a yellow marble

○ ○ ○
○ ○

↳ $P(o) = \frac{3}{5}$ → 1st Event → Orange marble has been removed

○ ○
○ ○

→ $P(y/o) = \frac{2}{4} = \frac{1}{2}$ ⇒ 2nd Event ⇒ Remove the Yellow Marble

$\downarrow$ 1st $\downarrow$ 2nd
 $P(o \text{ and } y) = P(o) * P(y/o)$ → Conditional Probability

$= \frac{3}{5} * \frac{1}{2} = \frac{3}{10}$

$P(A \text{ and } B) = P(A) * P(B/A)$

Figure 4: Naive Bayes Intuition

12 K-Nearest Neighbors (KNN)

Intuition: KNN is a non-parametric, "lazy learning" algorithm. Instead of learning an explicit mathematical model during training, it simply stores the dataset. When making a prediction for a new data point, it looks at the K closest data points (neighbors) in the feature space.

- **Classification:** The output is the majority class among the K nearest neighbors.
- **Regression:** The output is the average value of the K nearest neighbors.

12.1 Distance Metrics

To find the "nearest" neighbors, we calculate the distance between points.

Euclidean Distance (L2 Norm):

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (27)$$

Manhattan Distance (L1 Norm):

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (28)$$

*Note: Advanced data structures like **KD-Trees** or **Ball Trees** are often used to optimize the search for nearest neighbors in high-dimensional spaces, reducing the time complexity from $O(N)$ to $O(\log N)$.*

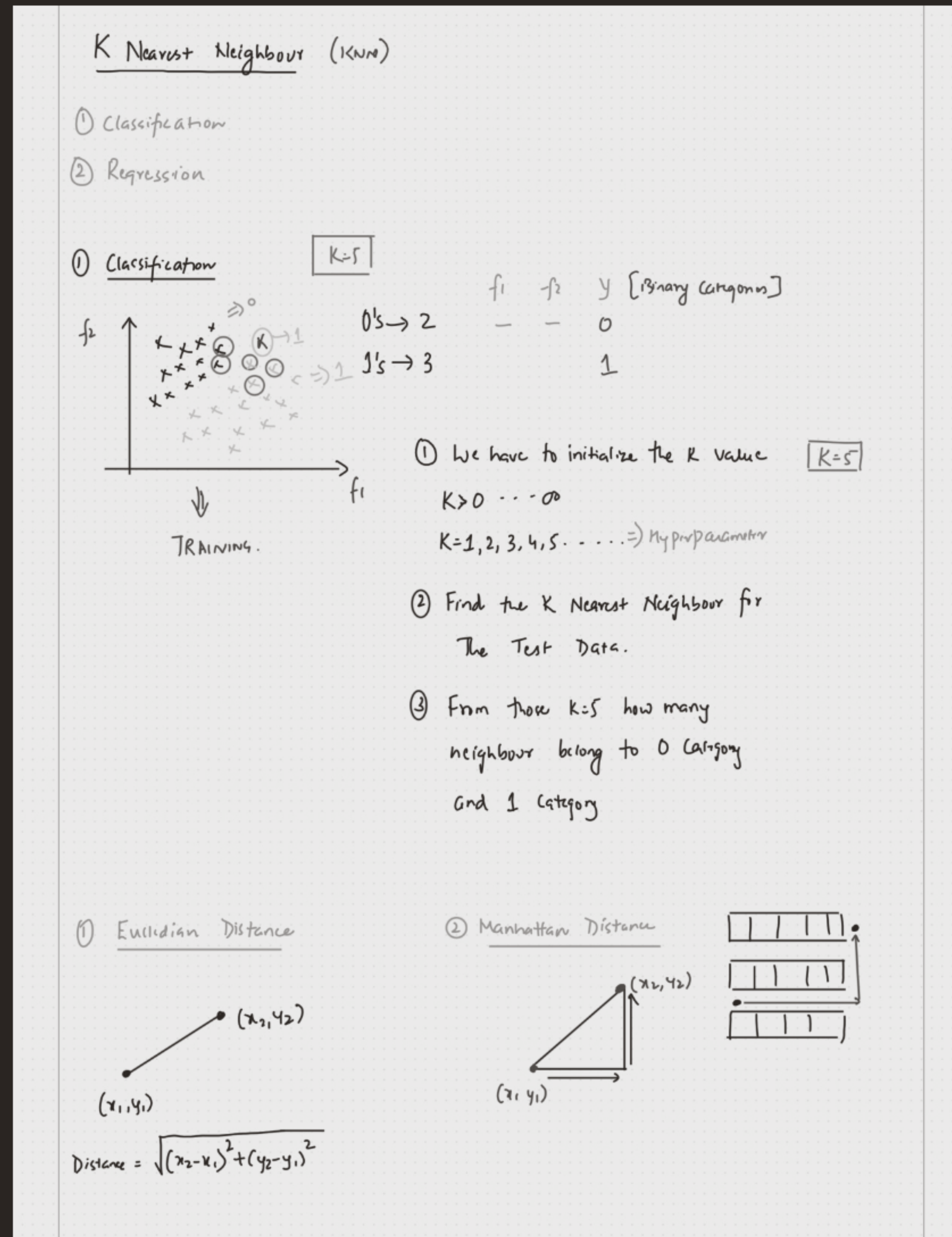


Figure 5: K-Nearest Neighbors Algorithm

13 Decision Trees

Intuition: A Decision Tree splits the dataset into smaller subsets using a tree-like structure of if-else decision rules. The goal is to maximize the homogeneity (purity) of the resulting child nodes. It can be used for both Classification and Regression, but it is highly prone to Overfitting unless **pruned** (restricting max depth).

13.1 Splitting Criteria & Purity Metrics

To decide the best feature to split on at each node, we measure the impurity of the data. The goal is to go from impure splits to pure splits. Lower impurity means better splits.

1. Entropy: Measures disorder. It peaks at 1.0 (maximum impurity) when classes are equally split. It is computationally heavier due to logarithms, making it better suited for **smaller datasets**.

$$\text{Entropy } H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-) \quad (29)$$

2. Gini Impurity (Used mainly by CART algorithm): Measures the probability of misclassification. It peaks at 0.5 when classes are evenly split. It is computationally faster and is generally preferred for **larger**

datasets.

$$\text{Gini} = 1 - \sum_{i=1}^C (p_i)^2 \quad (30)$$

13.2 Information Gain

Information Gain is the metric used to actually decide the best split. It calculates how much "impurity" is removed by splitting the data on a specific feature. The tree evaluates the gain for all available features and splits on the one that yields the highest value.

$$\text{Gain}(S, f) = \text{Impurity}(\text{Root}) - \sum_{v \in \text{values}(f)} \frac{|S_v|}{|S|} \times \text{Impurity}(S_v) \quad (31)$$

13.3 Handling Numerical / Continuous Features

To split on a continuous numerical column (e.g., Salary or Age):

1. The tree **sorts** all the numerical values in the feature.
2. It creates potential "thresholds" (midpoints between consecutive sorted values).
3. It calculates the Information Gain for every *single threshold* (e.g., ≤ 2.3 vs > 2.3) to find the optimal split.

Note: Because it requires sorting and exhaustive threshold checking, processing continuous numerical variables on millions of records significantly increases Time Complexity ($\uparrow\uparrow$).

13.4 Decision Tree Regression (CART)

When predicting a continuous output variable (like Salary) instead of a categorical class:

- **Splitting Metric:** Instead of Gini or Entropy, the tree evaluates splits using **Variance Reduction** (based on Mean Squared Error). It seeks to minimize the variance of the target values within the child nodes.
- **Variance Formula:** $\text{Var} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$
- **Variance Reduction:** $\text{Var}(\text{Root}) - \sum (W_i \times \text{Var}(\text{Child}_i))$
- **Leaf Node Prediction:** The final prediction output at any leaf node is simply the **Average (Mean)** of all the actual target values that fall into that specific leaf.

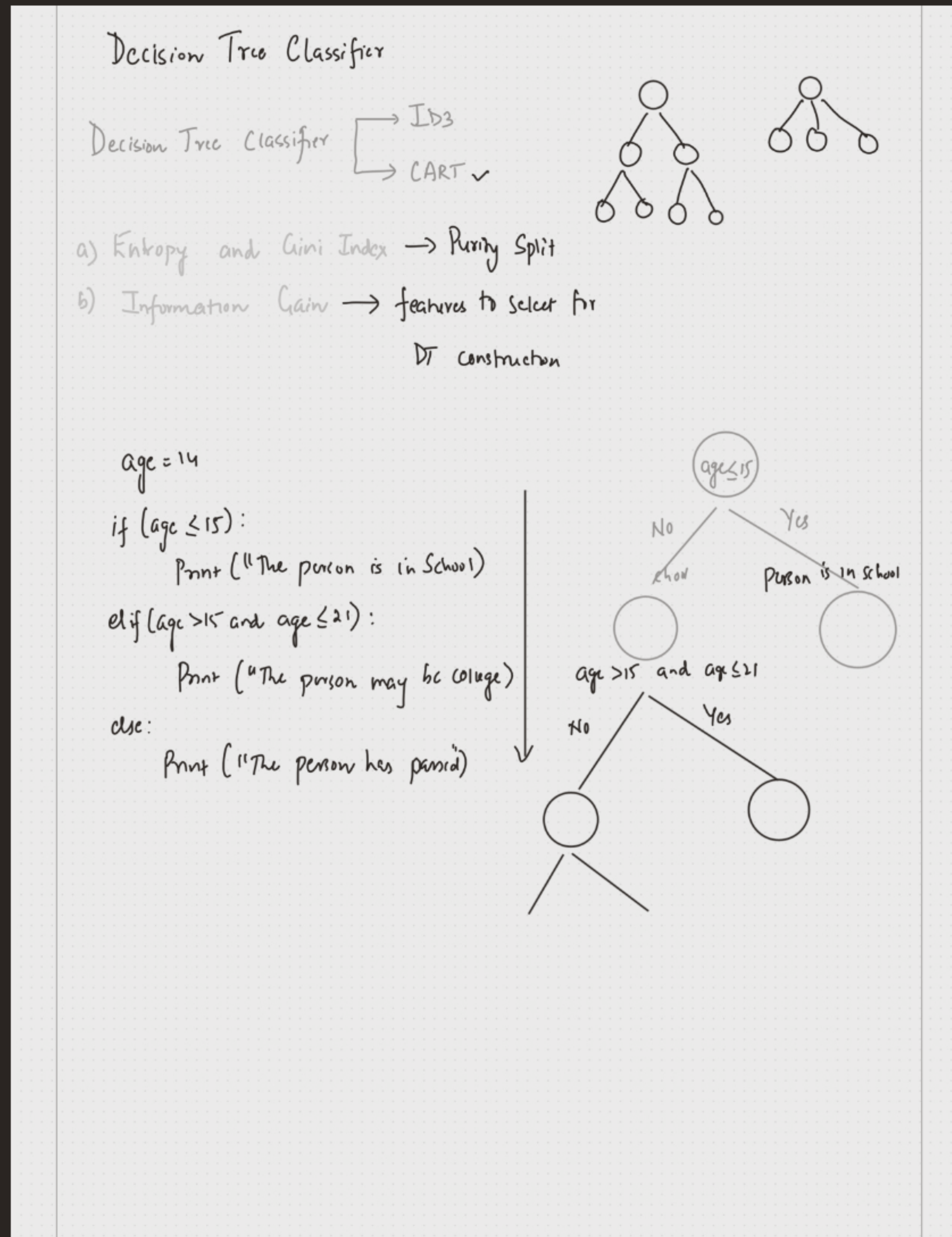


Figure 6: Decision Tree Splitting

14 Random Forest

Intuition: An ensemble learning method based on **Bagging** (Bootstrap Aggregating). It builds multiple Decision Trees using different random subsets of the training data (both rows and features) and aggregates their predictions to improve accuracy and drastically reduce Overfitting.

- **Classification:** Aggregation is done via **Majority Voting**.
- **Regression:** Aggregation is done via **Averaging**.

14.1 Key Concepts

1. Feature Randomness: At each split, only a random subset of features is considered, ensuring the trees are de-correlated.

2. Out-Of-Bag (OOB) Error: Since each tree is trained on a bootstrapped sample (roughly 63% of the data), the remaining 37% (OOB data) can be used to validate the model without needing a separate validation set.

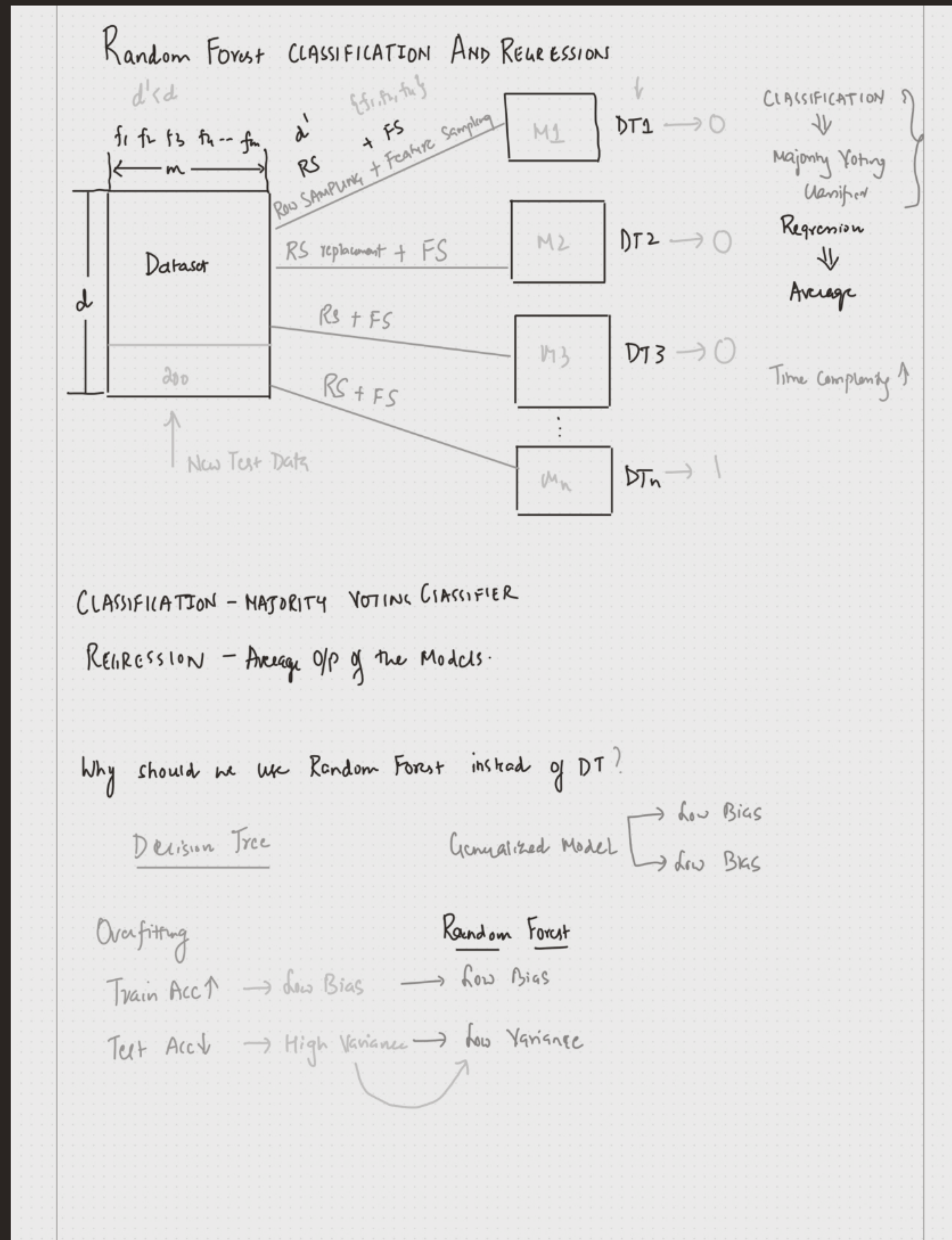


Figure 7: Random Forest Architecture

15 Boosting Algorithms

Intuition: Unlike Bagging where trees are built independently, Boosting builds models **sequentially**. Each new tree attempts to correct the errors (misclassifications or residuals) made by the previous sequence of trees.

15.1 AdaBoost (Adaptive Boosting)

Uses "Decision Stumps" (trees with a max depth of 1). After each stump is evaluated, it increases the weights of incorrectly classified instances so the next stump pays more attention to them.

$$\text{Amount of Say (Weight of Stump): } \alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (32)$$

Where ϵ_t is the total error of the stump.

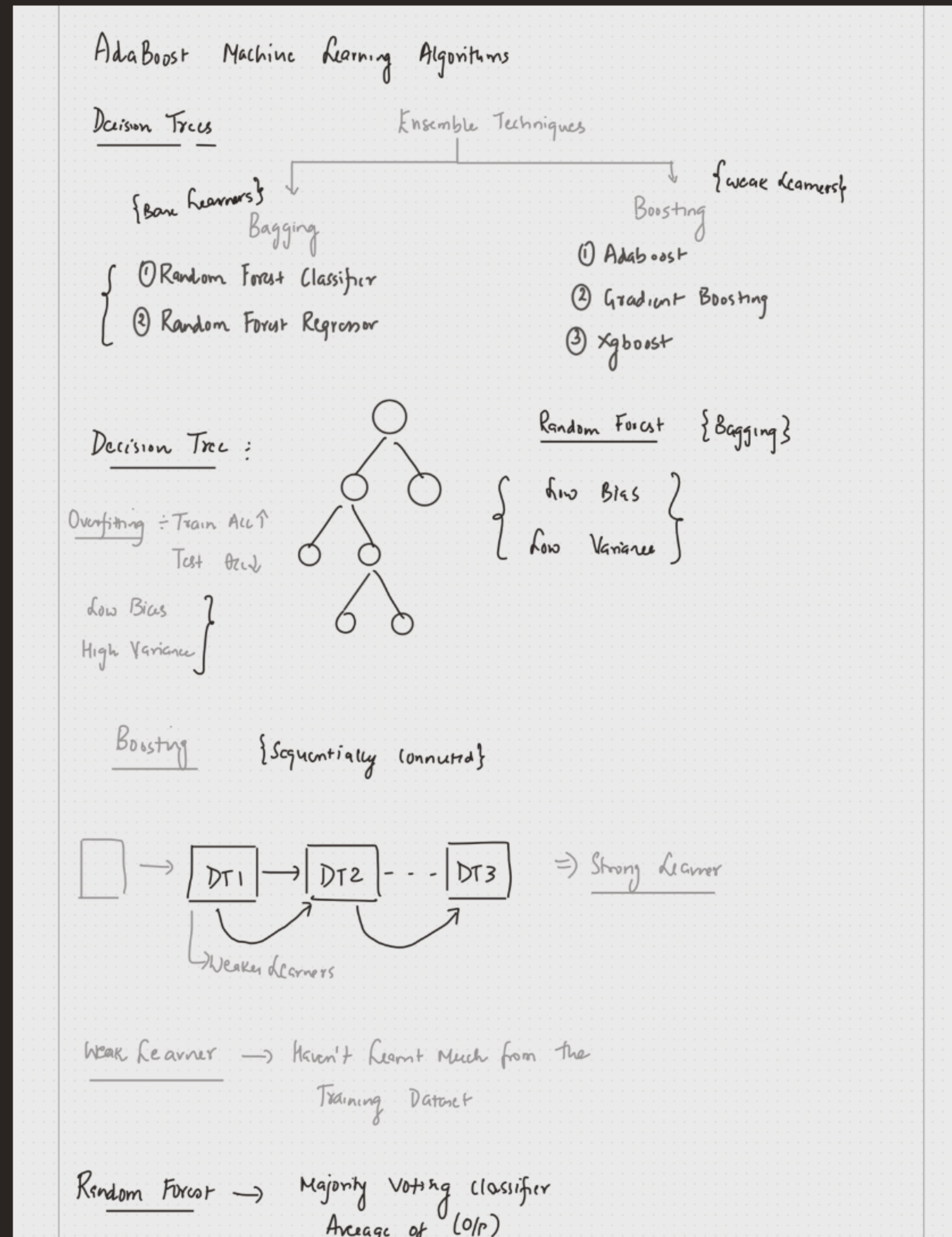


Figure 8: AdaBoost Weight Updating

15.2 Gradient Boosting

Instead of updating instance weights, Gradient Boosting fits subsequent trees to the **Pseudo-Residuals** (the gradients of the loss function) of the previous predictions.

$$\text{Residual Update: } r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad (33)$$

15.3 XGBoost (Extreme Gradient Boosting)

An optimized, highly scalable implementation of Gradient Boosting.

- Includes **L1 and L2 Regularization** to prevent Overfitting.
- Uses hardware optimization (cache awareness) and parallel processing.
- Has built-in handling for missing values.

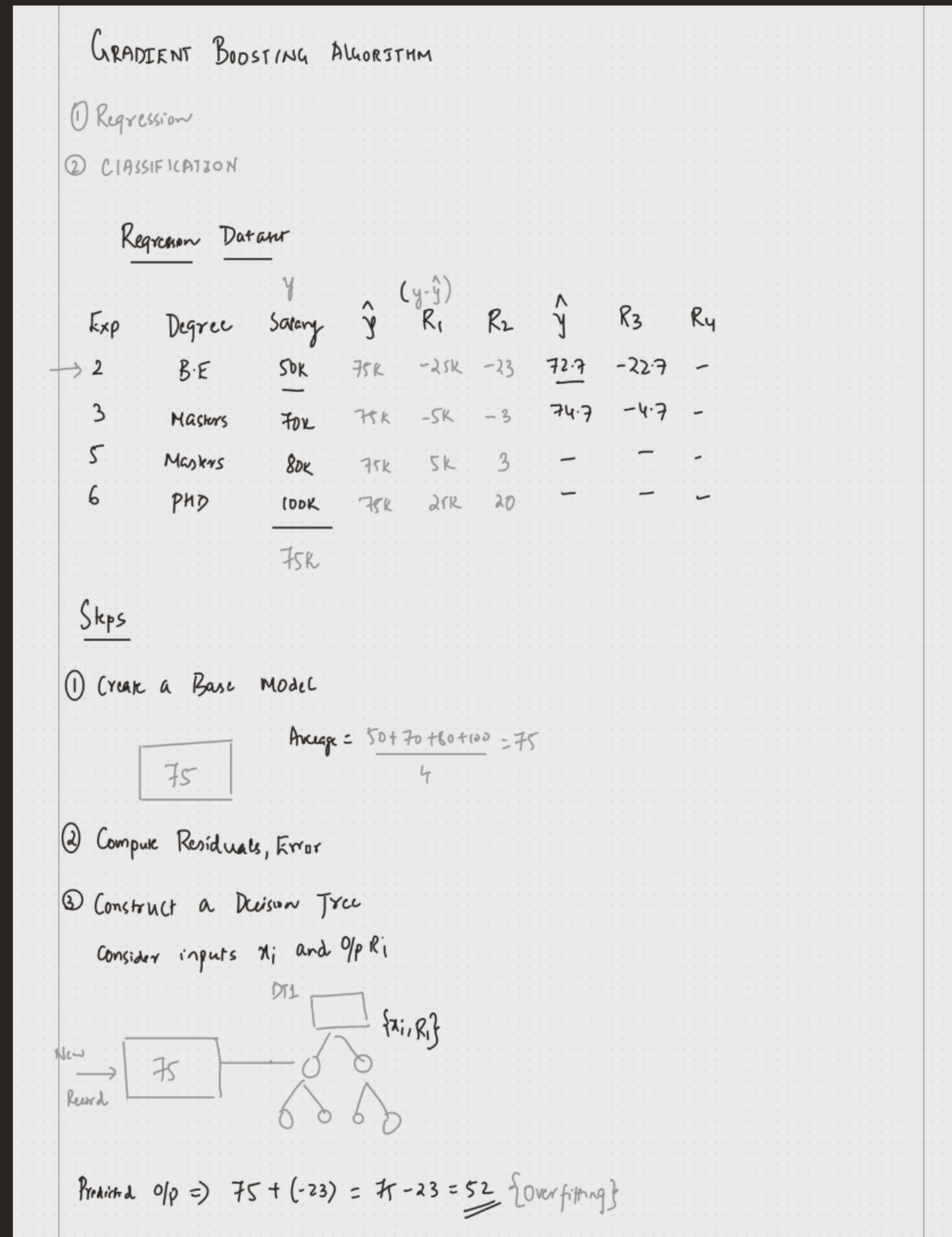


Figure 9: Gradient Boosting Residual Fitting

16 Unsupervised Learning: Dimensionality Reduction

16.1 Principal Component Analysis (PCA)

Intuition: PCA reduces the dimensionality of a dataset by projecting it onto new, orthogonal axes called **Principal Components**. These components are ranked by the amount of **variance** they capture from the original data, allowing us to drop less important dimensions with minimal information loss.

Mathematical Steps:

1. Standardize the data.
2. Compute the **Covariance Matrix**:

$$\Sigma = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^T \quad (34)$$

3. Compute the Eigenvectors (v) and Eigenvalues (λ) of the Covariance Matrix: $\Sigma v = \lambda v$.
4. Sort eigenvectors by their eigenvalues in descending order and select the top k .

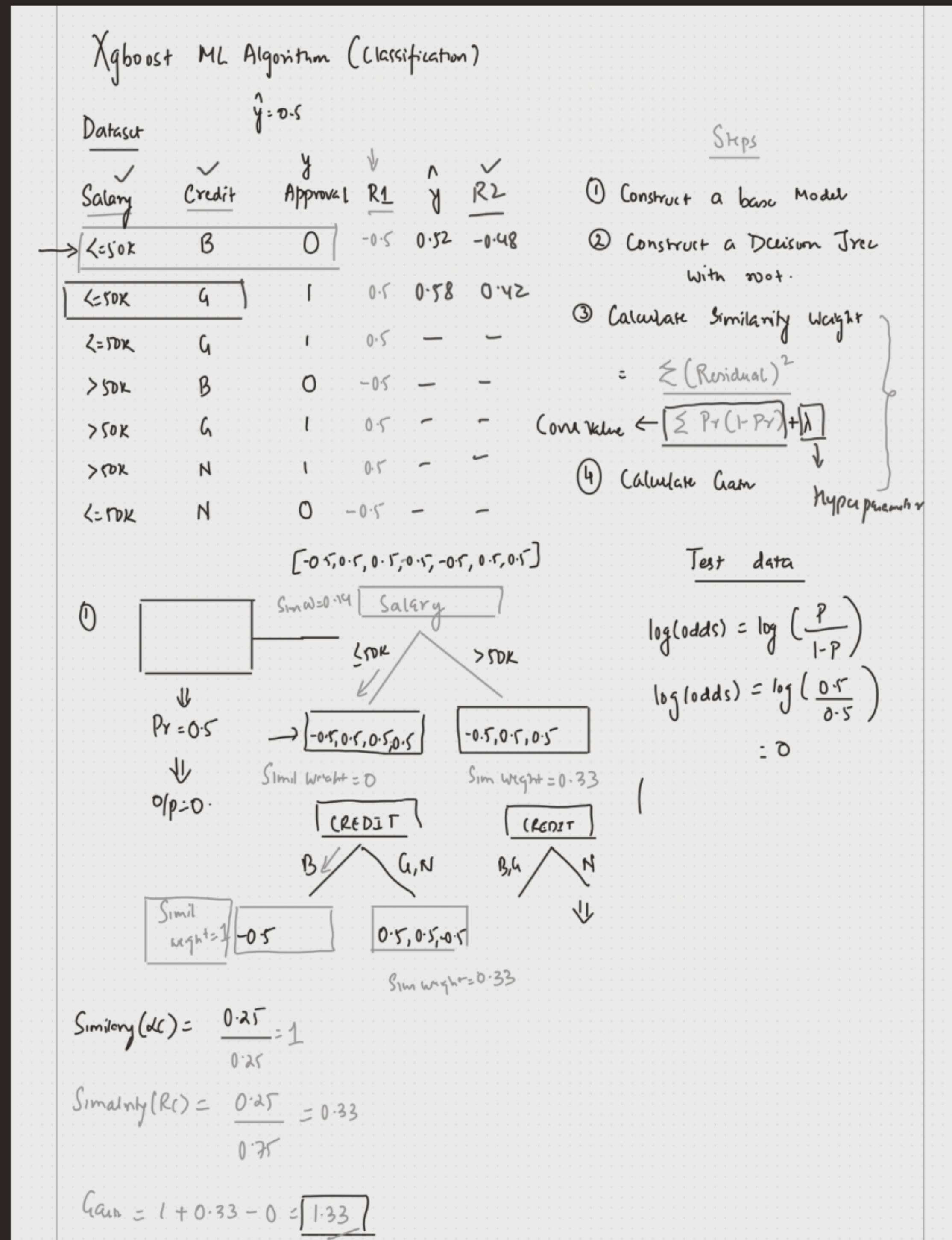


Figure 10: XGBoost Architecture

17 Unsupervised Learning: Clustering

17.1 K-Means Clustering

Intuition: Partitions data into K distinct, non-overlapping clusters. It iteratively assigns each point to the nearest centroid, then recalculates the centroid's position until convergence.

Objective (Minimize WCSS - Within-Cluster Sum of Squares):

$$\text{WCSS} = \sum_{j=1}^K \sum_{i \in C_j} \|x_i - \mu_j\|^2 \quad (35)$$

Note: The "Elbow Method" is used to find the optimal K by plotting WCSS against different K values.

17.2 Hierarchical Clustering

Intuition: Builds a hierarchy of clusters using a **Dendrogram**.

- **Agglomerative (Bottom-Up):** Each point starts as its own cluster, and the closest pairs merge step-

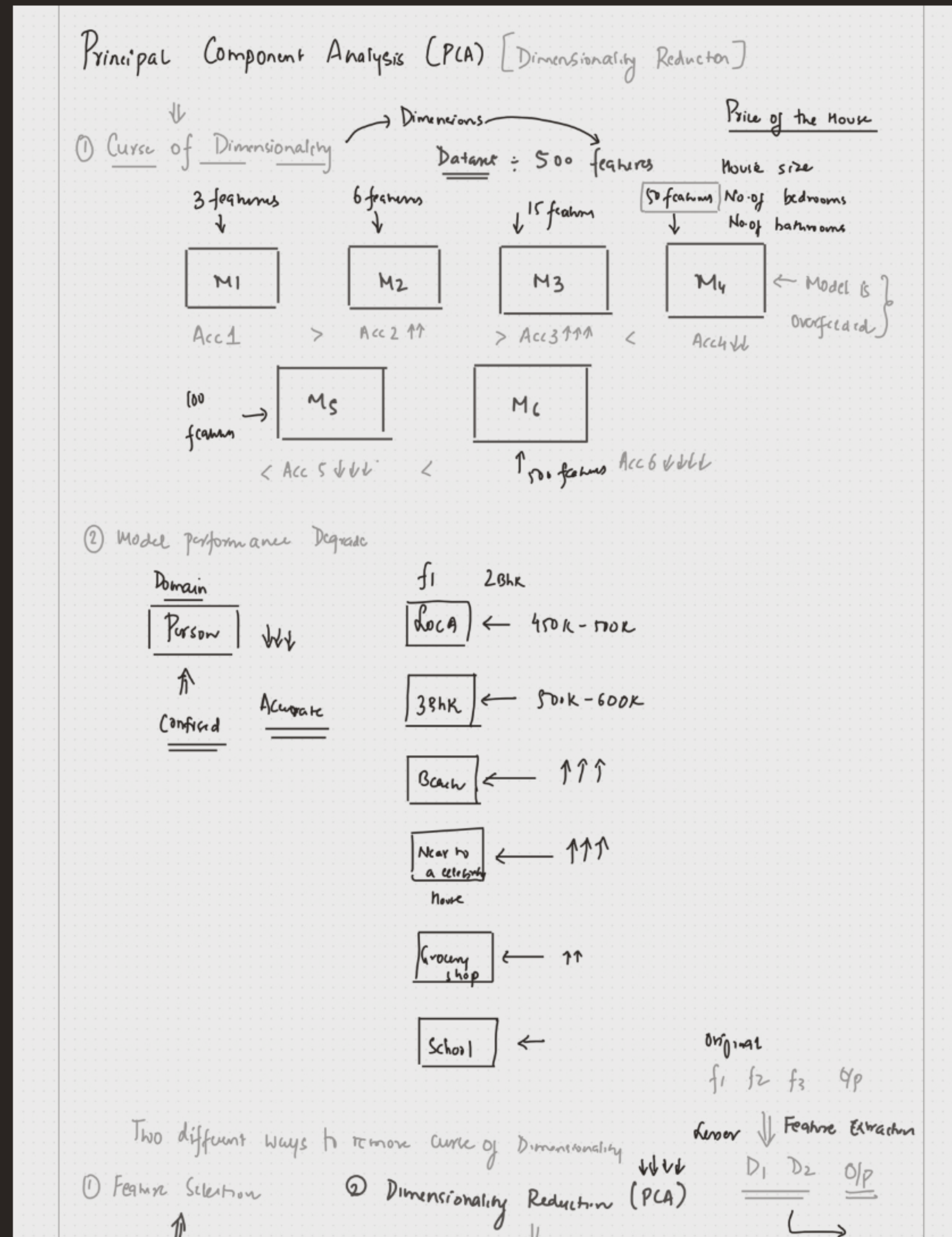


Figure 11: Principal Component Analysis

by-step.

- **Linkage Criteria:** Determines the distance between clusters (Single, Complete, Average, Ward's method).

17.3 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Intuition: Groups points that are closely packed together (many neighbors) and marks points in low-density regions as outliers (noise). It does not require specifying the number of clusters K beforehand.

- **Epsilon (ϵ):** The maximum distance between two points for them to be considered neighbors.
- **MinPts:** The minimum number of points required within ϵ distance to form a dense region (Core Point).

17.4 Silhouette Score (Clustering Validation)

Used to measure how similar an object is to its own cluster compared to other clusters. Values range from -1 to 1 (where 1 is best).

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (36)$$

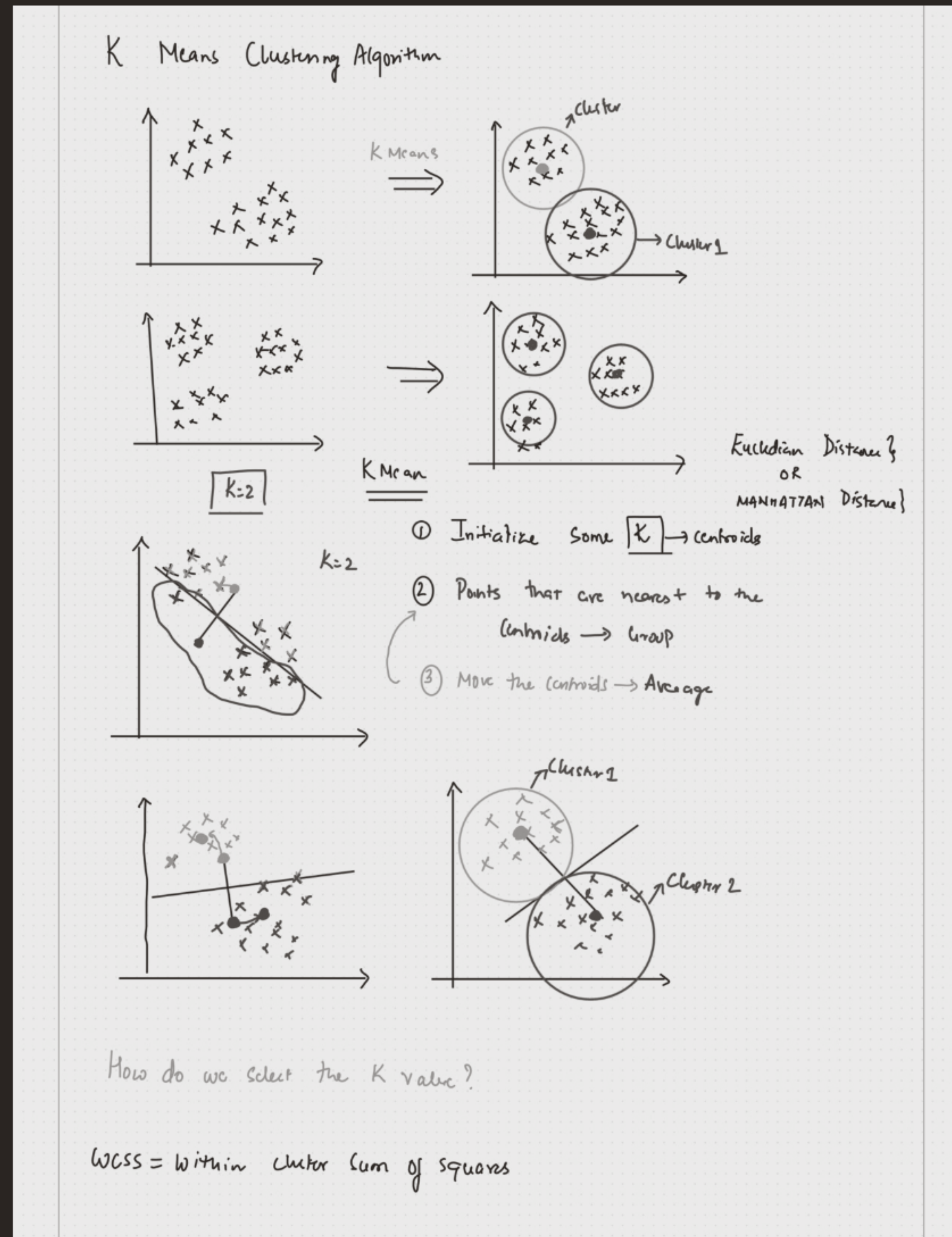


Figure 12: K-Means Clustering

18 Anomaly Detection

18.1 Isolation Forest

Intuition: Assumes that anomalies are "few and different". It builds random decision trees by randomly selecting features and split values. Since anomalies are isolated quickly, they have notably **shorter path lengths** from the root of the tree compared to normal points.

18.2 Local Outlier Factor (LOF)

Intuition: Measures the local density deviation of a given data point with respect to its neighbors. A point that has a substantially lower density than its neighbors is considered an outlier.

- $LOF \approx 1$: Similar density as neighbors (Normal).
- $LOF > 1$: Lower density than neighbors (Anomaly).

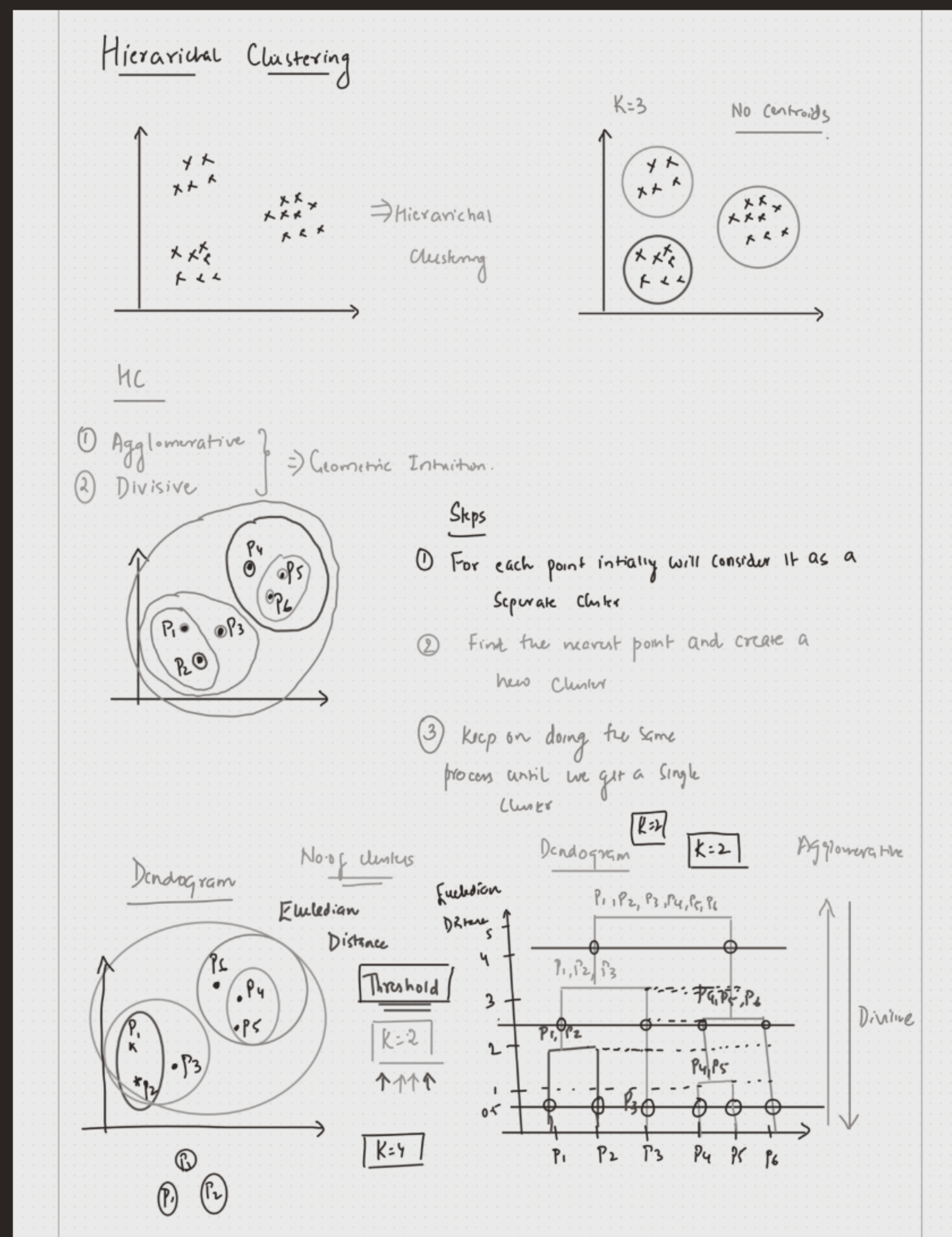
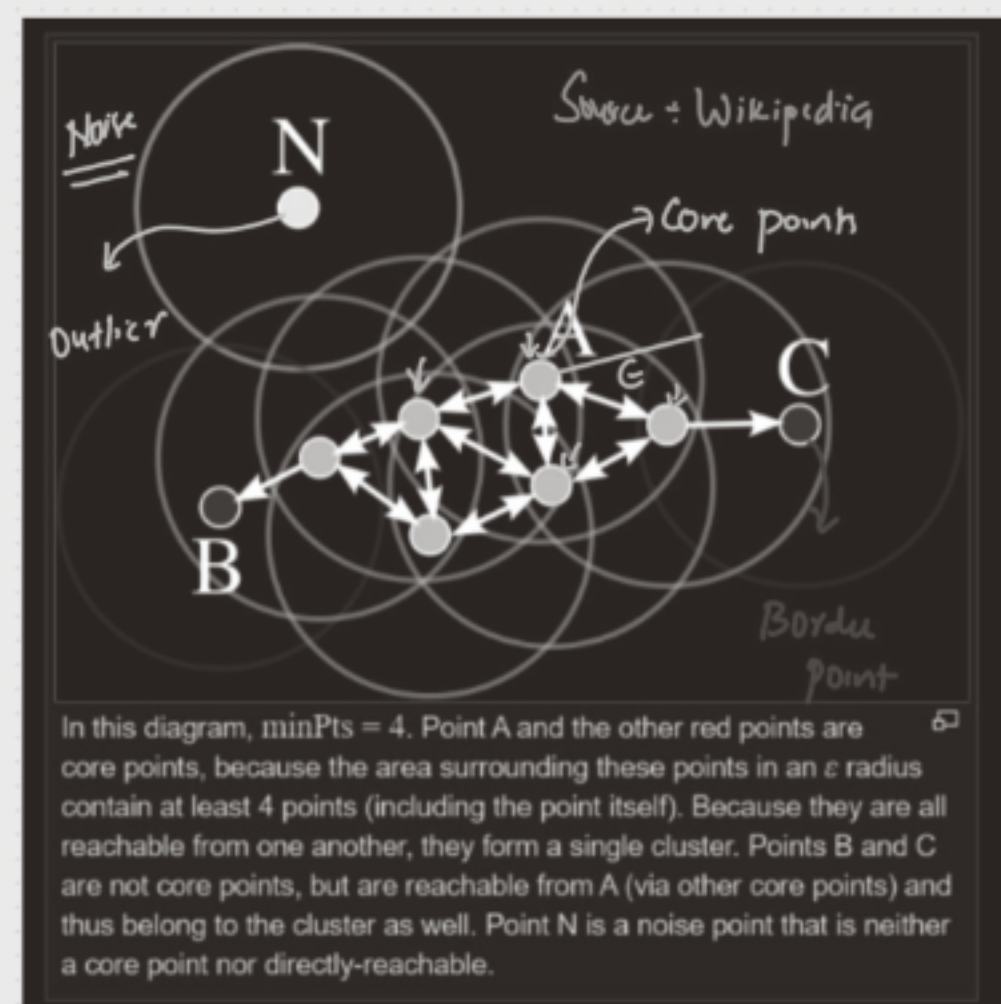


Figure 13: Hierarchical Clustering Dendrogram

DBSCAN CLUSTERING.



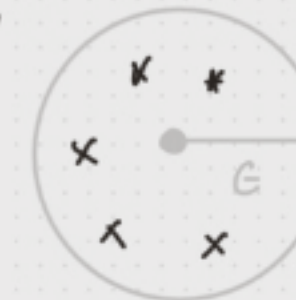
- → Core point
 - → border point
 - → Outlier
- } Non linear Clustering

$\text{minpts} = 4$ $\epsilon = \text{radius}$

Core point

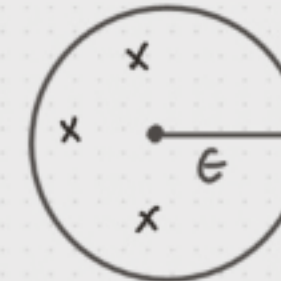
$\text{minpts} = 4$

① No. of points within the ϵ should be greater ≥ 4

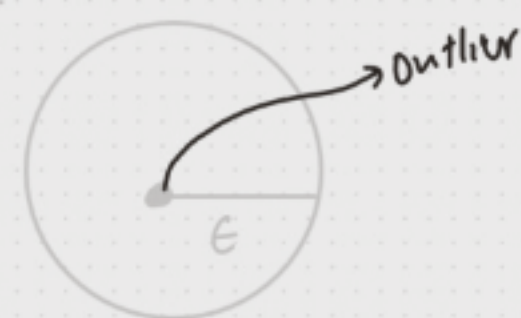


Border point

no. of data points within this radius will be less than minpts



Outlier (Noise)



Some Examples after we apply DBSCAN clustering

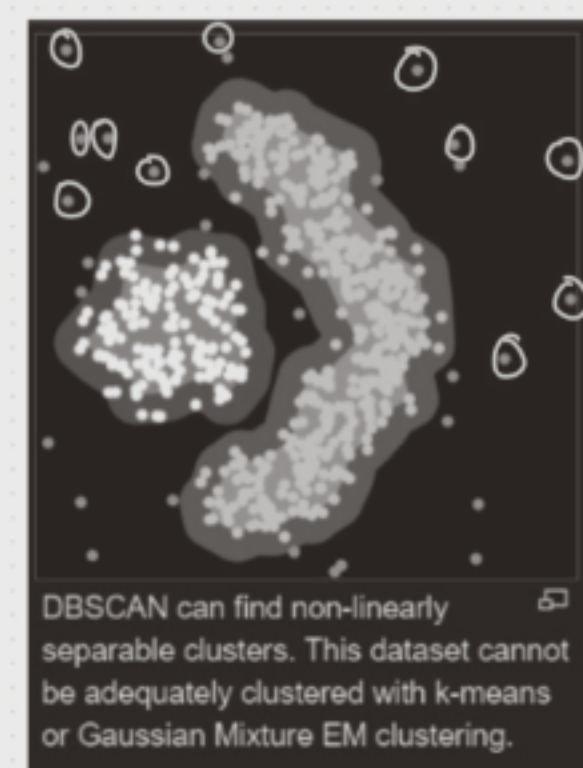


Figure 14: DBSCAN Clustering

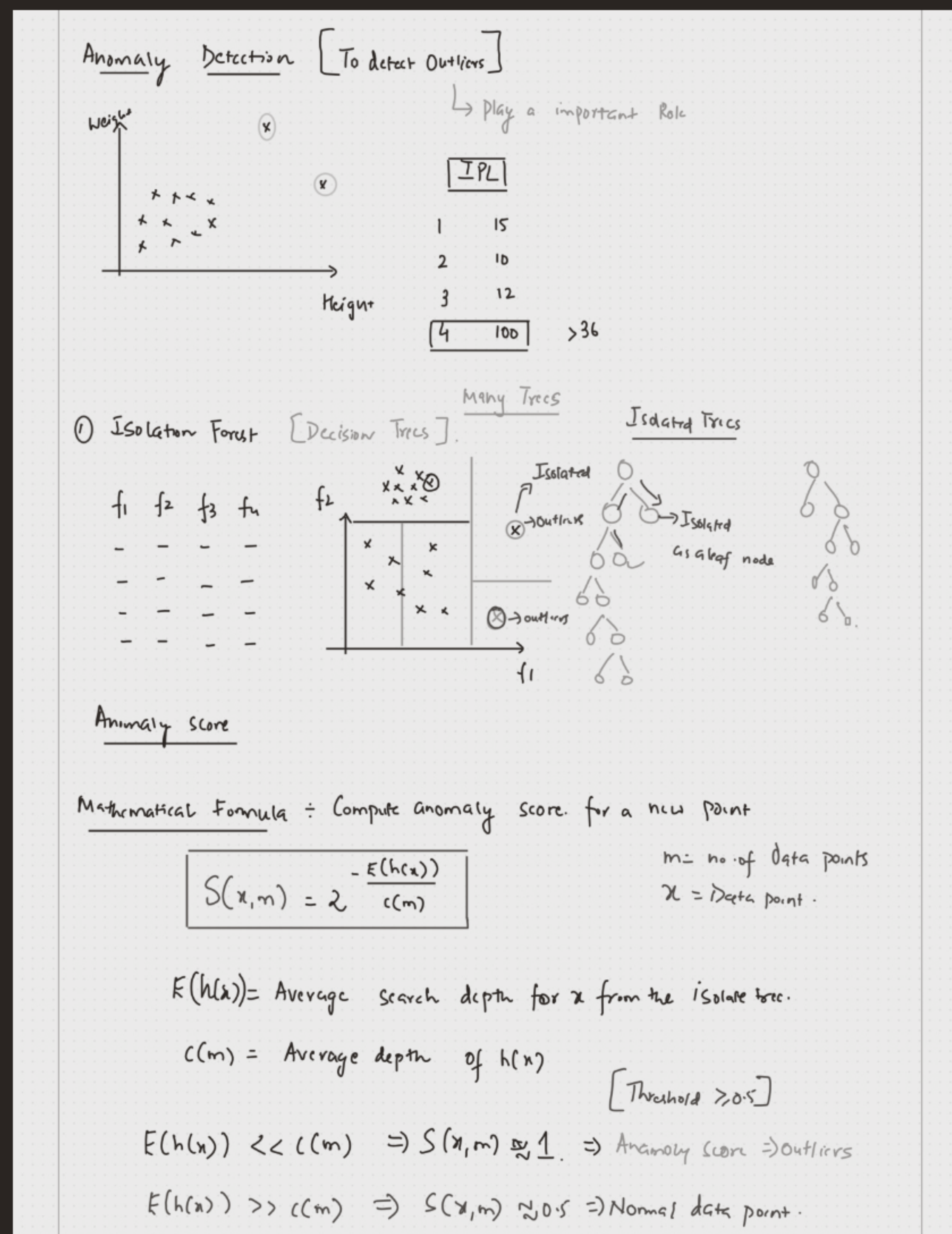


Figure 15: Isolation Forest Anomaly Detection

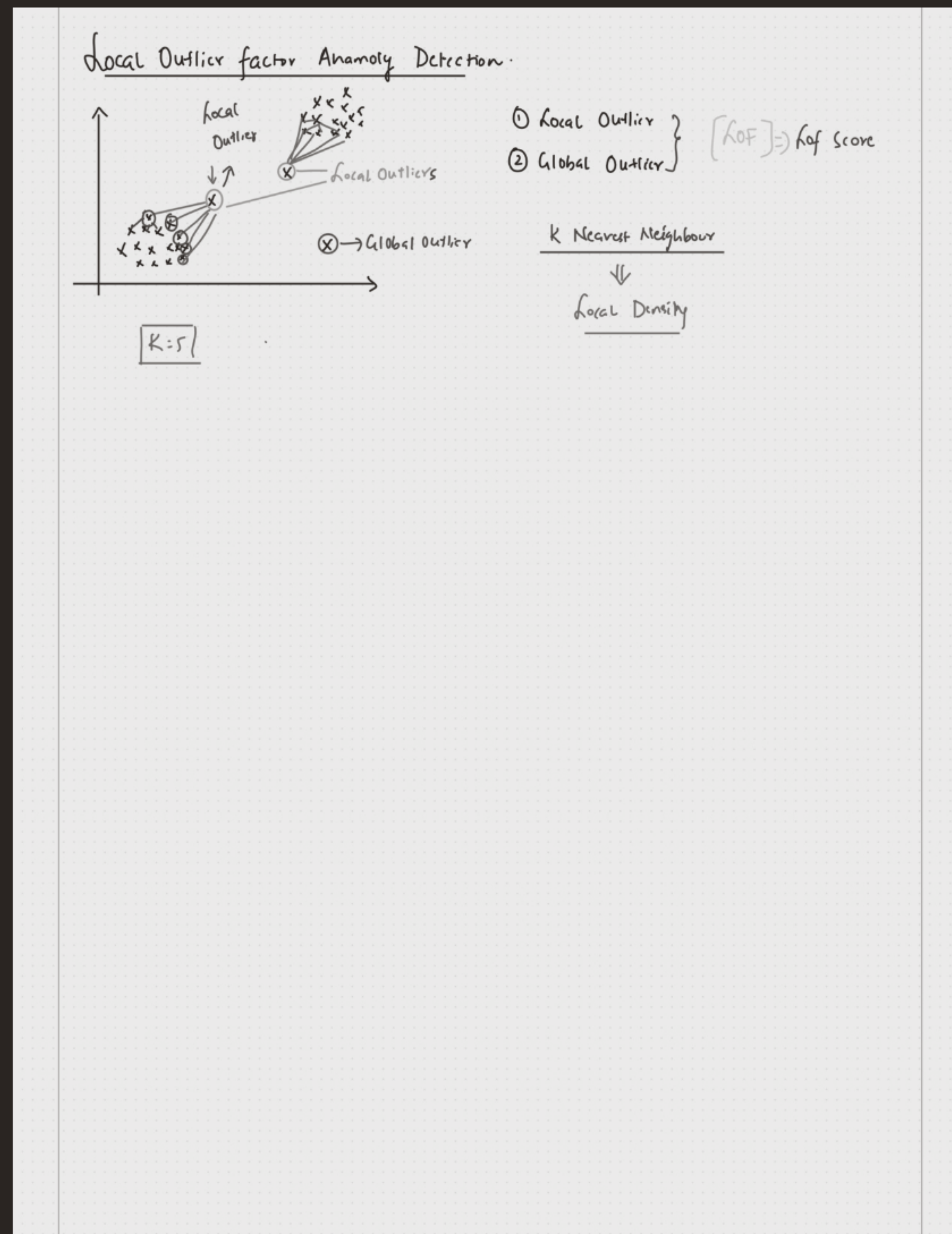


Figure 16: Local Outlier Factor Density Comparison